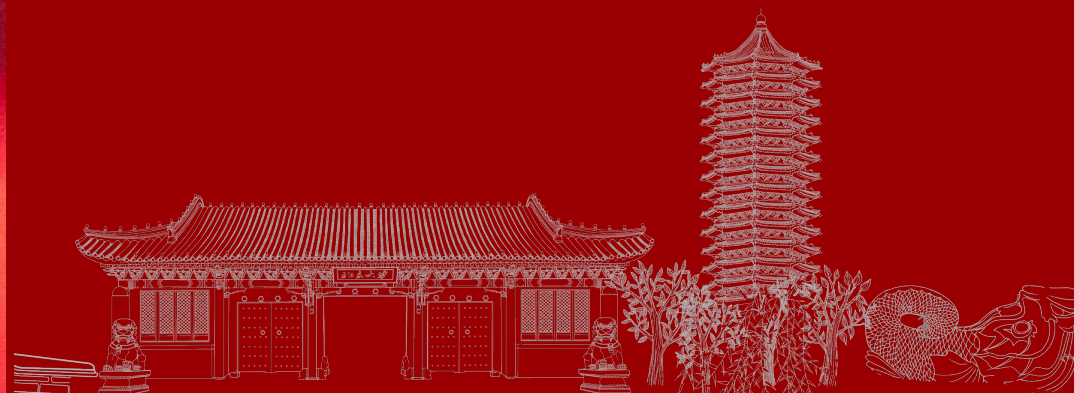


人工智能引论

16. 自然语言处理 -统计语言模型与词表示

授课教师 张辉帅

2026年5月11日



- 词袋模型 (Bag-of-Words Model)
- 朴素贝叶斯 (Naive Bayes) 模型
- 信息检索, tf-idf
- 词表示 (one-hot, word2vec)

回顾——垃圾邮件分类

输入: x = 邮件

TPAMI-2022-01- Review Request - Please Confirm or Decline



010101856ae810fc-8310f59f-52d5-4f36-abc1-cd94d3be2a92-00000...

on behalf of

Transactions on Pattern Analysis and Machine Intelligence <onbehalfof...

To: muhan@pku.edu.cn; Cc:

Sunday, January 1, 2023 at 9:17

RE: TPAMI TPAMI-2022-01-

Dear Dr. Zhang,

The above referenced manuscript has been submitted to Transactions on Pattern Analysis and Machine Intelligence. I am overseeing the review process of this paper and believe you are one of the experts best qualified to judge whether or not this paper should be accepted by the journal. Please note that if you have a relationship with the submitting author(s) that could be perceived as a conflict of interest, you should decline this invitation to review. Conflicts of interest could include, for instance, having the same affiliation as the author(s) or being a co-author of the submitting author(s) on a previous publication.

To ease the burden on those asked to perform the review and to accelerate the review process, I am including the abstract for this paper below. Please review it and let us know if you can participate in the peer review of TPAMI-2022-01-

Attention.

Paul Thurston <test@test.com>

Saturday, December 31, 2022 at 12:19

To: Recipients

This message appears to be Junk. Links and other functionality will not work.

Mark as Not Junk

Hello,

I hope this email wouldn't come to you as a surprise. Having gone through your profile, I am convinced and pleased to solicit your sincere and urgent assistance over this matter.

I am Barrister Paul Thurston, an Attorney to late Mr. Peter a local contractor with the Lagos State Government of Nigeria. My client Late was awarded a contract worth of One Hundred and Fifty-Five Million United States Dollars (155,000,000:00USD) by the Lagos state Government of Nigeria for the maintenance and construction of Nigerian National Petroleum corporation Pipeline at Ejigbo Power Depot in the states in 2018 and the contract was fully executed by my late client. My client has been paid One Hundred Million United States Dollars from the contract and he was to receive the balance payment of the contract, but unfortunately my client was involved in a fatal auto accident with his entire family on Sagamu Express Road where he eventually lost his life and his two kids and wife. This was on the 25th of November 2020.

Sequel to the death of my late client, there is every need to present a next of kin to Nigerian National Petroleum

输出: $y \in \{\text{正常邮件}, \text{垃圾邮件}\}$

目标: 训练模型 f , 使得 $f(x) = y$

回顾——垃圾邮件分类



北京大学
PEKING UNIVERSITY

训练数据 (Training Data)

(email1, not_spam)

(email2, spam)

(email3, spam)

.....

(email97, not_spam)

(email98, spam)

(email99, not_spam)

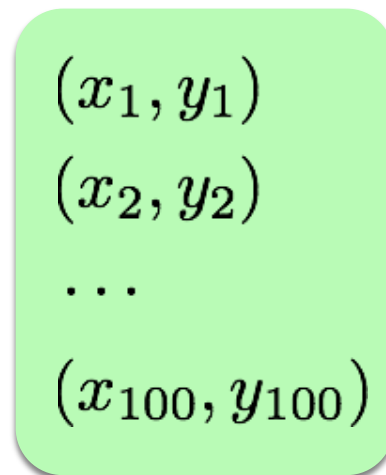
(email100, spam)

特征提取
(Feature extraction)



x_i : 特征向量 (feature vector)

y_i : 标签 (label)



模型训练
(Model Training)



Learning a
function f

$$y_i \approx f(x_i) \quad \forall i$$

x_i 词频向量

attention:	1
million:	2
dollars:	1
prize:	0
payment:	3
government:	1

...

$y_i = \{-1, 1\}$

词袋模型 (Bag-of-Words Model)

- **词袋模型** (BoW) : 不考虑文本中词与词之间的上下文关系, 仅仅只考虑所有词的权重, 而权重与词在文本中出现的频率有关。以词的统计直方图作为该文本 (文档/句子) 的特征
- 原始BoW模型的两要素:
 - ✓ 构建一个包含所有已知单词的字典
 - ✓ 统计文本中单词出现的频率
- 改变字典构建方法和单词出现情况统计策略可得到升级版本的BoW

x_i 词频向量

attention:	1
million:	2
dollars:	1
prize:	0
payment:	3
government:	1
...	

词袋模型 (Bag-of-Words Model)

- **词袋模型** (BoW) 构建实例

- 步骤1**: 数据收集

It was the best of times,
it was the worst of times,
it was the age of wisdom,
it was the age of foolishness,

From "A Tale of Two Cities"

- 整个语料库包括4行文本
- 每行是一个独立的“文档”

词袋模型 (Bag-of-Words Model)

• 词袋模型 (BoW) 构建实例

步骤2：字典构建

It was the best of times,
it was the worst of times,
it was the age of wisdom,
it was the age of foolishness,



字典
"it"
"was"
"the"
"best"
"of"
"times"
"worst"
"age"
"wisdom"
"foolishness"

- 把所有单词列出
- 去重，不考虑大小写和标点符号
- 字典包含10个词
- 整个语料库包括24个词

词袋模型 (Bag-of-Words Model)

• 词袋模型 (BoW) 构建实例

步骤3：生成文本特征向量

- ① It was the best of times,
- ② it was the worst of times,
- ③ it was the age of wisdom,
- ④ it was the age of foolishness,

提问：输入以下新文本，其特征向量是？

This is the best of times but also the worst of times.

[0, 0, 2, 1, 2, 2, 1, 0, 0, 0]

字典	①	②	③	④
“it”	1	1	1	1
“was”	1	1	1	1
“the”	1	1	1	1
“best”	1	0	0	0
“of”	1	1	1	1
“times”	1	1	0	0
“worst”	0	1	0	0
“age”	0	0	1	1
“wisdom”	0	0	0	0
“foolishness”	0	0	1	1

BoW模型的缺点：字典可能极大；文本向量稀疏；关键词的重要性未体现.....

文本情感分类



北京大学
PEKING UNIVERSITY

好评度

98%

尺寸适宜(2)

烹饪速度快(1)

真材实料(1)

工艺精美(1)

非常好用(1)



家电放心买 买贵双倍赔 (5.1-5.15)

全部评价(1000+)

晒图(62)

视频晒单(2)

追评(6)

好评(200+)

中评(2)

差评(2)

☐ 只看当前商品评价



★★★★★

大肚能容！因为要蒸菜蒸饭所以要添加个蒸锅，对比了好多牌子好多型号和实用性，个人感觉这个性价比比较适合，蒸笼高低都有方便，最后用过感觉没选错，值得入手！



★★★★★

蒸锅非常美观大方，试用了一下，煮水很快，满意好评。



★★★★★

这个宝贝非常好，是我想要的尺寸，材质也好。



★☆☆☆☆

锅盖滴水，蒸出来的馒头皱巴巴的，一点也不光，不知道蒸其他东西咋样，反正蒸馒头是确实不好用，当时就想买个蒸馒头的锅，看着很好，买回来一试，很失望！京东的商品现在有些挺让人失望的，感觉京东好像不是最开始的京东了，好多商品还蛮失望的，哎！！



★☆☆☆☆

用了没几次就坏了，销售说给换一个，不知道下个会不会再坏！

- 😊 "My grandson loved it! So much fun!"
- 😞 "Product broke after a few days."
- 😊 "One of the best games I've played in a long time."
- 😞 "Kind of cheap and flimsy, not worth it."

- 😊 "My grandson **loved** it! So much **fun**!"
- 😞 "Product **broke** after a few days."
- 😊 "One of the **best** games I've played in a long time."
- 😞 "Kind of **cheap** and **flimsy**, not worth it."

回顾:

BoW模型缺点: 字典可能极大; 文本向量稀疏; **关键词的重要性未体现**.....

朴素贝叶斯 (Naive Bayes) 模型

回顾:

1.7 贝叶斯 (Bayes) 公式

Bayes公式:

设 $B_1, B_2, \dots, B_n$ 构成一组完备事件组, $P(B_i) > 0$,

则对任一事件 A 有: $P(B_i|A) = \frac{P(B_i)P(A|B_i)}{\sum_{j=1}^n P(B_j)P(A|B_j)}$.

- 称 $P(B_i|A)$ 为**后验概率**
- 已知结果发生, 对导致结果发生的因素的可能性大小重新修正
- 贝叶斯 (Bayes) 公式体现了**执果求因**

全概率公式: $P(A) = \sum_{i=1}^n P(B_i)P(A|B_i)$ **由因求果**

例: 根据收集到的数据, 已知: $P(\text{男性}|\text{糖尿病}) = 0.2$, $P(\text{糖尿病}) = 0.05$, $P(\text{男性}) = 0.5$, 计算: $P(\text{糖尿病}|\text{男性})$?

解:

$$\begin{aligned} P(\text{糖尿病}|\text{男性}) &= \frac{P(\text{男性且患糖尿病})}{P(\text{男性})} \\ &= \frac{P(\text{男性}|\text{糖尿病}) \cdot P(\text{糖尿病})}{P(\text{男性})} \\ &= \frac{0.2 \times 0.05}{0.5} \\ &= 0.02 \end{aligned}$$

朴素贝叶斯 (Naive Bayes) 模型

贝叶斯准则

$$P(b|a) = \frac{P(a|b)P(b)}{P(a)}$$

$$P(\text{正面}) \leftarrow P(\text{😊})$$

$$P(\text{负面}) \leftarrow P(\text{😞})$$

朴素贝叶斯 (Naive Bayes) 模型

"My grandson loved it!"



待求解: $P(\text{😊} | \text{"my grandson loved it"})$

不考虑上下文关系 

$P(\text{😊} | \text{"my", "grandson", "loved", "it"})$



$$\frac{P(\text{"my", "grandson", "loved", "it"} | \text{😊}) P(\text{😊})}{P(\text{"my", "grandson", "loved", "it"})}$$

朴素贝叶斯 (Naive Bayes) 模型

"My grandson loved it!"



待求解: $P(\text{😊} | \text{"my grandson loved it"})$

不考虑上下文关系 

$P(\text{😊} | \text{"my", "grandson", "loved", "it"})$



等比例于

$P(\text{"my", "grandson", "loved", "it"} | \text{😊})P(\text{😊})$

朴素贝叶斯 (Naive Bayes) 模型

"My grandson loved it!"



待求解: $P(\text{😊} | \text{"my grandson loved it"})$

不考虑上下文关系 

$P(\text{😊} | \text{"my", "grandson", "loved", "it"})$

假设词间相互条件独立  则近似等比例于

$$P(\text{😊})P(\text{"my"} | \text{😊})P(\text{"grandson"} | \text{😊})P(\text{"loved"} | \text{😊})P(\text{"it"} | \text{😊})$$

朴素贝叶斯 (Naive Bayes) 模型

待求解: $P(\text{😊} | \text{"my grandson loved it"})$



近似等比例于

$$P(\text{😊})P(\text{"my"} | \text{😊})P(\text{"grandson"} | \text{😊})P(\text{"loved"} | \text{😊})P(\text{"it"} | \text{😊})$$

$$P(\text{😊}) = \frac{\text{正面样本数量}}{\text{样本总数量}}$$

$$P(\text{"loved"} | \text{😊}) = \frac{\text{正面样本中"loved"出现的总次数}}{\text{正面样本中所有单词出现的总次数}}$$

朴素贝叶斯 (Naive Bayes) 模型

待求解: $P(\text{😊} | \text{"my grandson loved it"})$

近似等比例于

$$P(\text{😊})P(\text{"my"} | \text{😊})P(\text{"grandson"} | \text{😊})P(\text{"loved"} | \text{😊})P(\text{"it"} | \text{😊})$$

😊	😞
0.49	0.51

$$P(\text{😊} | \text{"my grandson loved it"}) \propto 0.00014112$$

	😊	😞
my	0.30	0.20
grandson	0.01	0.02
loved	0.32	0.08
it	0.30	0.40

朴素贝叶斯 (Naive Bayes) 模型

待求解: $P(\text{😞} | \text{"my grandson loved it"})$

⚡ 近似等比例于

$$P(\text{😞})P(\text{"my"} | \text{😞})P(\text{"grandson"} | \text{😞})P(\text{"loved"} | \text{😞})P(\text{"it"} | \text{😞})$$

😄	😞
0.49	0.51

$P(\text{😄} | \text{"my grandson loved it"})$
近似等比例于0.00014112

$P(\text{😞} | \text{"my grandson loved it"})$
近似等比例于0.00006528

	😄	😞
my	0.30	0.20
grandson	0.01	0.02
loved	0.32	0.08
it	0.30	0.40

朴素贝叶斯 (Naive Bayes) 模型

待求解: $P(\text{😞} | \text{"my grandson loved it"})$

近似等比例于

$$P(\text{😞})P(\text{"my"} | \text{😞})P(\text{"grandson"} | \text{😞})P(\text{"loved"} | \text{😞})P(\text{"it"} | \text{😞})$$

😄	😞
0.49	0.51

$$P(\text{😄} | \text{"my grandson loved it"}) \approx 0.6837$$

$$P(\text{😞} | \text{"my grandson loved it"}) \approx 0.3163$$

	😄	😞
my	0.30	0.20
grandson	0.01	0.02
loved	0.32	0.08
it	0.30	0.40

朴素贝叶斯 (Naive Bayes) 模型

待求解: $P(\text{😊} | \text{"my grandson loved it"})$

存在问题!

近似等比例于

$$P(\text{😊})P(\text{"my"} | \text{😊})P(\text{"grandson"} | \text{😊})P(\text{"loved"} | \text{😊})P(\text{"it"} | \text{😊})$$

😊	😞
0.49	0.51

$P(\text{😊} | \text{"my grandson loved it"})$
近似等比例于 **0.00000000**

$P(\text{😞} | \text{"my grandson loved it"})$
近似等比例于 0.00006528

	😊	😞
my	0.30	0.20
grandson	0.00	0.02
loved	0.32	0.08
it	0.30	0.40

朴素贝叶斯 (Naive Bayes) 模型

解决方案:

- 加法平滑(additive smoothing)

- ✓ 给每个单词在正负样本中的数量都加上一个固定值 α

$$P(\text{"loved"}|\text{😊}) = \frac{\text{正面样本中loved出现的总次数} + \alpha}{\text{正面样本的总词数} + \text{词表大小} \times \alpha}$$

- 拉普拉斯平滑(Laplace smoothing)

- ✓ 令 $\alpha = 1$

例: $P(\text{"grandson"}|\text{😊}) = \frac{0+1}{500+10 \times 1} = \frac{1}{510}$
这里假设了词表大小为10

信息检索：tf-idf

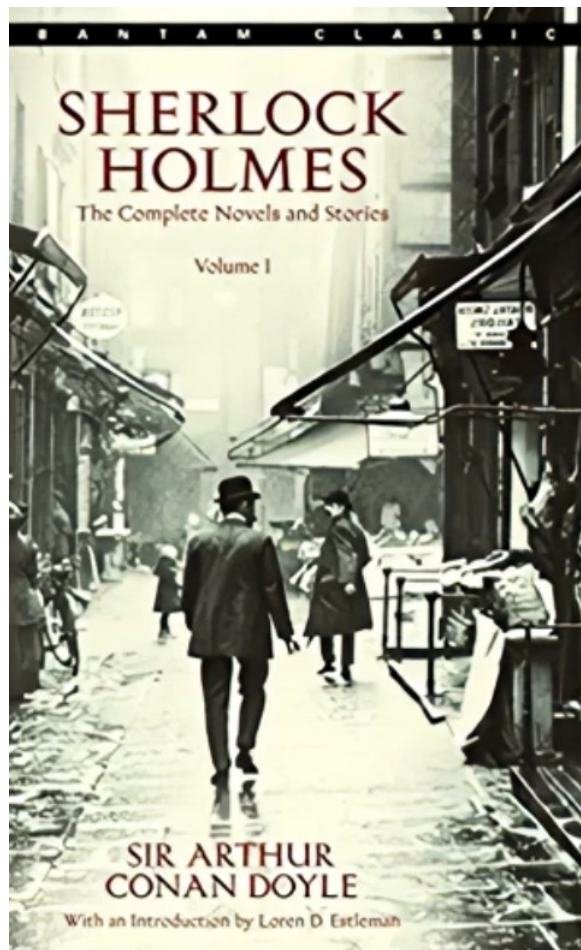


- 如何根据用户的输入找到相关文档？
 - ✓ 文档：一本书、一个章节、一段话、一句话等
- 如何找到给定文档的主题和关键词？
 - ✓ 通过词频统计衡量某个词对该文档的重要性
- **词频** (term frequency, **tf**):
 - ✓ 给定文档中某个词出现的频率

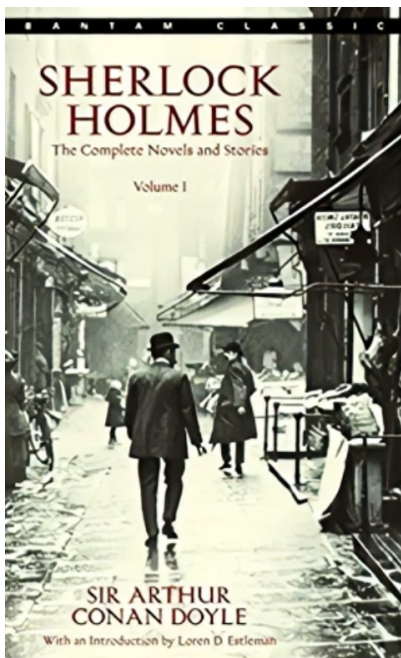
$$tf = \frac{n}{N} \quad \text{或} \quad tf = \log_{10}(n + 1)$$

n : 某个词在文档中出现的次数

N : 文档中所有词出现的次数之和



信息检索: tf-idf



league.txt

the: 460.0000
and: 271.0000
i: 264.0000
a: 239.0000
of: 224.0000

gloria scott.txt

the: 430.0000
and: 273.0000
of: 220.0000
a: 203.0000
i: 188.0000

carbuncle.txt

the: 463.0000
of: 233.0000
a: 208.0000
and: 199.0000
i: 188.0000

squires.txt

the: 508.0000
of: 206.0000
and: 169.0000
to: 168.0000
a: 152.0000

twisted.txt

the: 493.0000
a: 275.0000
and: 270.0000
i: 238.0000
of: 234.0000

blaze.txt

the: 641.0000
of: 242.0000
and: 242.0000
a: 242.0000
to: 238.0000

ritual.txt

the: 482.0000
of: 255.0000
and: 216.0000
to: 200.0000
i: 200.0000

engineer.txt

the: 431.0000
i: 313.0000
and: 250.0000
a: 233.0000
to: 215.0000

problem.txt

the: 427.0000
i: 231.0000
to: 209.0000
of: 191.0000
and: 187.0000

crooked.txt

the: 438.0000
and: 204.0000
of: 199.0000
i: 184.0000
a: 175.0000

coronet.txt

the: 466.0000
i: 356.0000
to: 270.0000
and: 238.0000
a: 213.0000

copper.txt

the: 485.0000
i: 330.0000
and: 275.0000
to: 256.0000
a: 238.0000

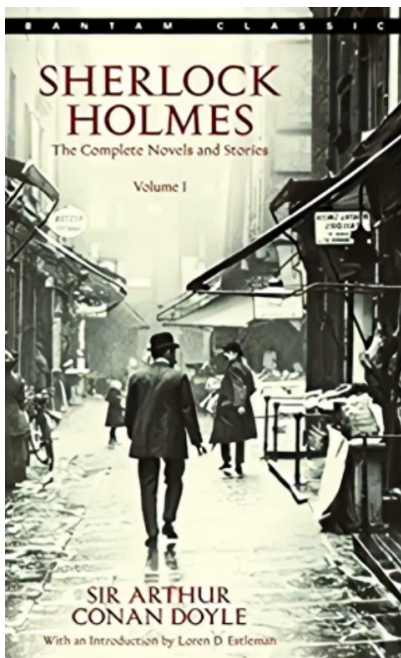
功能词(function words):

am, by, do, is, which, with....

实义词(content words):

man, horse, hat....

信息检索: tf-idf



league.txt

holmes: 53.0000
man: 26.0000
little: 25.0000
see: 23.0000
wilson: 22.0000

squires.txt

holmes: 53.0000
man: 35.0000
cunningham: 31.00
inspector: 29.0000
colonel: 25.0000

ritual.txt

musgrave: 26.00
old: 26.0000
brunton: 23.00
man: 22.0000
two: 20.0000

crooked.txt

barclay: 34.0000
man: 27.0000
colonel: 20.0000
know: 18.0000
room: 18.0000

gloria scott.txt

man: 28.0000
know: 24.0000
trevor: 23.0000
came: 19.0000
hudson: 17.0000

twisted.txt

man: 31.0000
holmes: 29.0000
clair: 27.0000
little: 21.0000
face: 20.0000

engineer.txt

time: 25.0000
little: 25.0000
colonel: 20.00
door: 19.0000
came: 18.0000

coronet.txt

coronet: 27.0000
holmes: 27.0000
man: 27.0000
think: 26.0000
house: 23.0000

去掉功能词:

am, by, do, is, which, with....

留下**实义词**(content words):

man, horse, hat....

carbuncle.txt

man: 39.0000
holmes: 38.0000
hat: 27.0000
see: 27.0000
goose: 26.0000

blaze.txt

holmes: 51.0000
horse: 50.0000
colonel: 44.0000
straker: 38.0000
inspector: 21.00

problem.txt

holmes: 28.0000
watson: 22.0000
man: 21.0000
moriarty: 20.00
away: 15.0000

copper.txt

holmes: 43.0000
rucastle: 38.0000
little: 37.0000
miss: 36.0000
man: 34.0000

- **逆文档频率** (inverse document frequency, **idf**):

- ✓ 衡量某个词在语料库的所有文档中的罕见程度

$$idf = \log_{10} \frac{D}{1 + d}$$

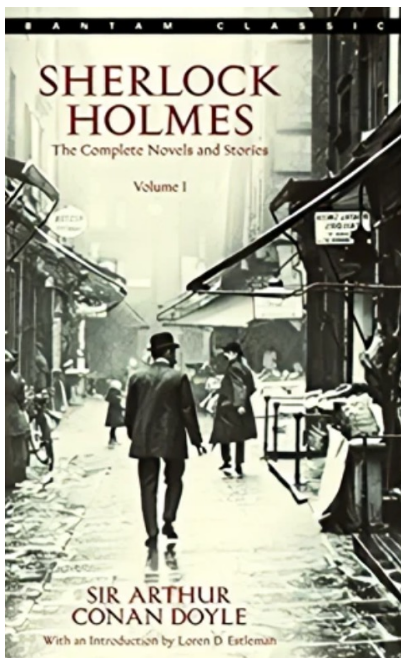
D : 语料库中包含的文档总数量

d : 语料库中出现某个词的文档数量

- **tf-idf**: $tfidf = tf \times idf$

- ✓ 用tf-idf值可以弱化常见词，保留重要的词。若某个词在某个文档中是高频词，在整个语料中又是低频出现，那么这个词将具有高tf-idf值，它对这篇文档来说，就是关键词，或主题词。

信息检索: tf-idf



league.txt

wilson: 42.8100
league: 37.6220
merryweather: 36.5343
jones: 33.4897
assistant: 32.9193

squires.txt

cunningham: 94.3802
alec: 57.8459
acton: 45.6678
william: 31.5063
colonel: 31.3191

ritual.txt

brunton: 70.0240
musgrave: 61.1358
ritual: 35.2706
hurlstone: 27.4007
butler: 21.5263

crooked.txt

barclay: 103.5138
colonel: 25.0553
aldershot: 18.8110
nancy: 18.2671
regiment: 14.1083

gloria scott.txt

trevor: 70.0240
beddoes: 33.4897
hudson: 24.3964
prendergast: 21.3117
boat: 18.8110

twisted.txt

clair: 82.2021
neville: 57.8459
lascar: 36.5343
opium: 25.8651
whitney: 24.3562

engineer.txt

hydraulic: 30.4452
stark: 27.4007
eyford: 27.4007
colonel: 25.0553
engineer: 24.3562

coronet.txt

coronet: 82.2021
arthur: 44.6761
gems: 39.5788
holder: 29.8481
snow: 23.5138

✓ 用tf-idf值弱化
常见词，保留
重要的词。

carbuncle.txt

goose: 61.1358
geese: 51.7569
horner: 39.5788
ryder: 36.5343
peterson: 33.4897

blaze.txt

straker: 115.6919
colonel: 55.1216
horse: 54.9306
trainer: 51.7569
moor: 48.7124

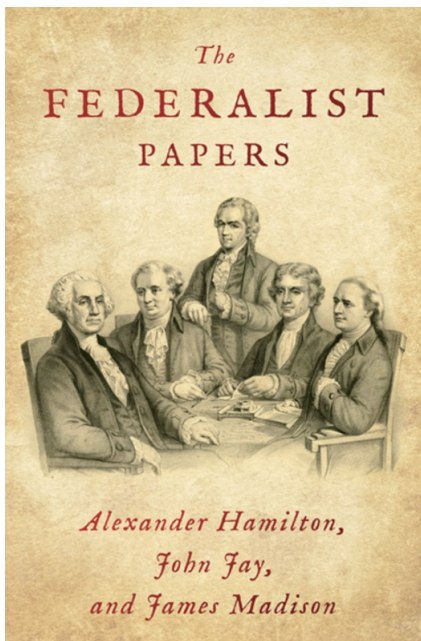
problem.txt

moriarty: 60.8904
professor: 28.2165
spray: 18.2671
rock: 18.2671
meiringen: 15.2226

copper.txt

rucastle: 115.6919
hunter: 51.7303
toller: 45.6678
copper: 25.8651
beeches: 25.8651 31

信息检索: tf-idf



✓ 用tf-idf值弱化
常见词，保留
重要的词。

53.disputed.txt

elections: 25.2503
knowledge: 18.8099
annual: 14.9804
biennial: 12.2254
year: 11.6904

51.disputed.txt

sects: 11.2485
weaker: 11.2271
department: 11.1045
rigorously: 8.8853
angels: 8.8853

77.hamilton.txt

nomination: 17.4772
senate: 14.1213
president: 11.3814
convening: 10.0321
council: 9.7869

52.disputed.txt

elections: 23.4467
biennial: 15.2818
parliaments: 13.3280
ii: 10.0321
septennial: 10.0321

69.hamilton.txt

governor: 47.1540
president: 30.3503
king: 24.4101
treason: 15.2818
britain: 11.0953

72.hamilton.txt

exclusion: 22.4923
he: 14.8577
his: 13.3773
ill: 10.7003
station: 8.9817

54.disputed.txt

slaves: 36.7844
property: 21.4982
representation: 17.7044
inhabitants: 15.3794
southern: 14.1661

40.madison.txt

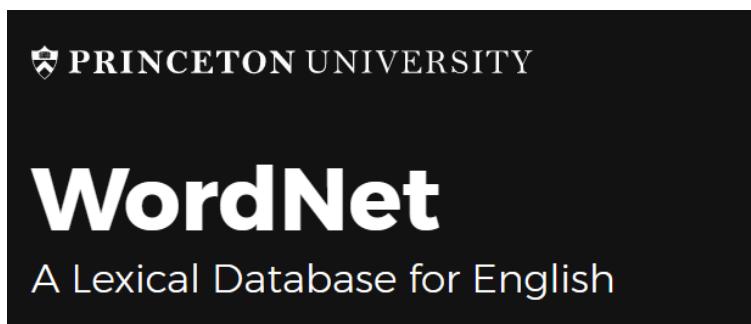
confederation: 23.5288
alterations: 22.4707
annapolis: 17.7706
confirmed: 17.7706
recommendatory: 17.7706

60.hamilton.txt

landed: 18.3381
discrimination: 15.2818
preference: 13.1439
class: 10.3213
partiality: 9.9870

WordNet

- **WordNet**: An Electronic Lexical Database
- 按照单词的意义组成一个“单词的网络”
- WordNet 3.0收录117798名词, 11529动词, 22479形容词, 4481副词。名词, 动词, 形容词和副词各自被组织成一个同义词的网络, 每个同义词集合都代表一个基本的语义概念, 并且这些集合之间也由各种关系连接。



WordNet

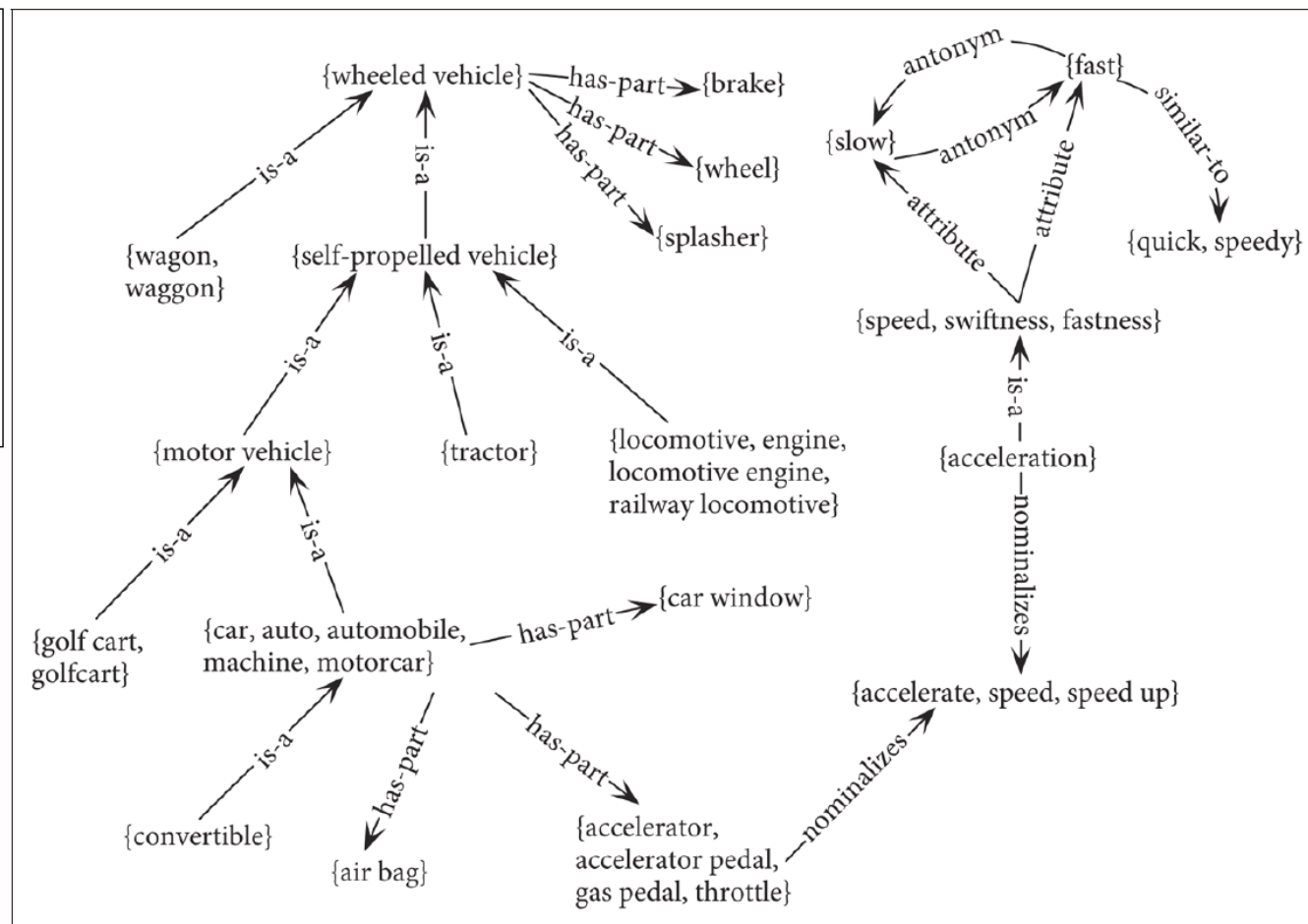


The noun “bass” has 8 senses in WordNet.

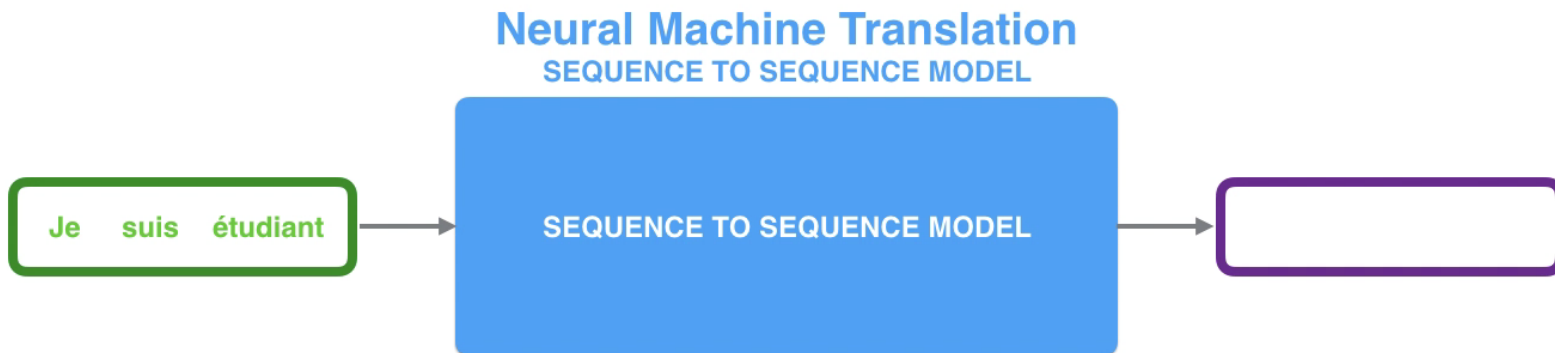
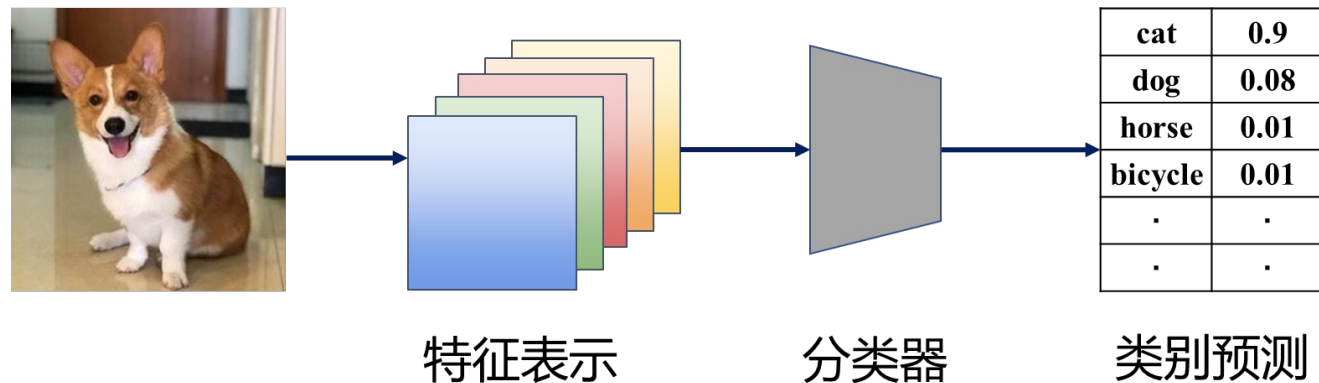
1. bass¹ - (the lowest part of the musical range)
2. bass², bass part¹ - (the lowest part in polyphonic music)
3. bass³, basso¹ - (an adult male singer with the lowest voice)
4. sea bass¹, bass⁴ - (the lean flesh of a saltwater fish of the family Serranidae)
5. freshwater bass¹, bass⁵ - (any of various North American freshwater fish with lean flesh (especially of the genus Micropterus))
6. bass⁶, bass voice¹, basso² - (the lowest adult male singing voice)
7. bass⁷ - (the member with the lowest range of a family of musical instruments)
8. bass⁸ - (nontechnical name for any of numerous edible marine and freshwater spiny-finned fishes)

bass³, basso (an adult male singer with the lowest voice)

=> singer, vocalist, vocalizer, vocaliser
 => musician, instrumentalist, player
 => performer, performing artist
 => entertainer
 => person, individual, someone...
 => organism, being
 => living thing, animate thing,
 => whole, unit
 => object, physical object
 => physical entity
 => entity



词表示 (Word Representation)



- 如何将词用某些向量来表达?

词表示 (Word Representation)

- 独热表示(one-hot representation)

"He wrote a book."

he: [1, 0, 0, 0]

wrote: [0, 1, 0, 0]

a: [0, 0, 1, 0]

book: [0, 0, 0, 1]

词表示 (Word Representation)

- 独热表示(one-hot representation)

✓ **问题1**: 字典若很大, 则词向量很长很稀疏, 例如字典中有50000个单词

"He wrote a book."

he: [1, 0, 0, 0, 0, 0, 0, 0, 0.....]

wrote: [0, 1, 0, 0, 0, 0, 0, 0, 0.....]

a: [0, 0, 1, 0, 0, 0, 0, 0, 0.....]

book: [0, 0, 0, 1, 0, 0, 0, 0, 0.....]

50000

词表示 (Word Representation)

- 独热表示(one-hot representation)

✓ **问题2**: 仅将词符号化, 不包含任何语义信息, 没有考虑词间的相关性

"He wrote a book."

"He authored a novel."

wrote [0, 1, 0, 0, 0, 0, 0, 0, 0]

authored [0, 0, 0, 0, 1, 0, 0, 0, 0]

≠

book [0, 0, 0, 0, 0, 0, 1, 0, 0]

novel [0, 0, 0, 0, 0, 0, 0, 0, 1]

≠

词表示 (Word Representation)

- 分布式表示 (distributed representation)

✓ **理论基础**: 上下文相似的词, 其语义也相似。词的语义由其上下文决定

“tejuino”



C1: A bottle of ____ is on the table.

C2: Everybody likes ____.

C3: Don't have ____ before you drive.

C4: We make ____ out of corn.

词表示 (Word Representation)

- 分布式表示 (distributed representation)

✓ **理论基础**: 上下文相似的词, 其语义也相似。词的语义由其上下文决定

C1: A bottle of ____ is on the table.

C2: Everybody likes ____.

C3: Don't have ____ before you drive.

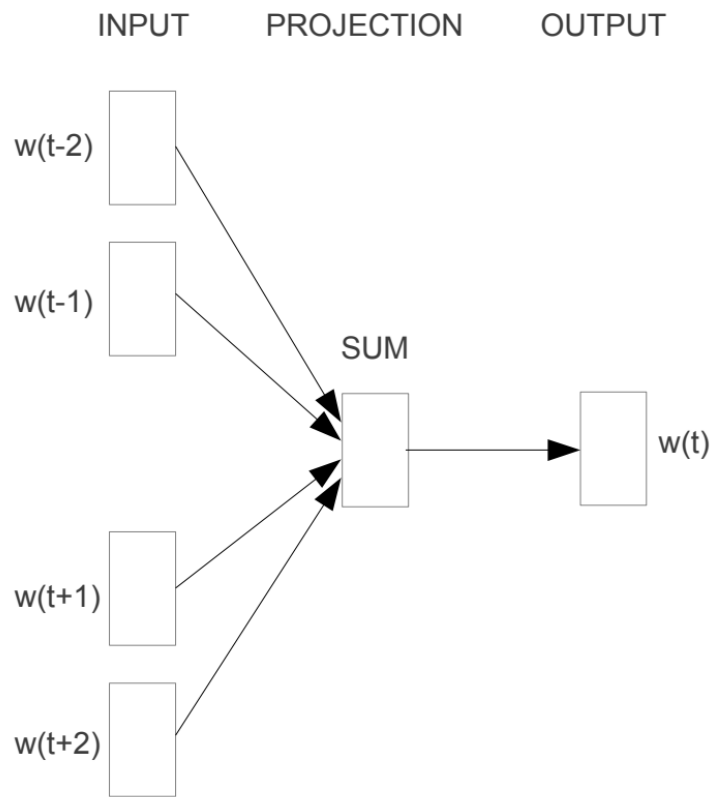
C4: We make ____ out of corn.

	C1	C2	C3	C4
tejuino	1	1	1	1
loud	0	0	0	0
motor-oil	1	0	0	0
tortillas	0	1	0	1
choices	0	1	0	0
wine	1	1	1	0

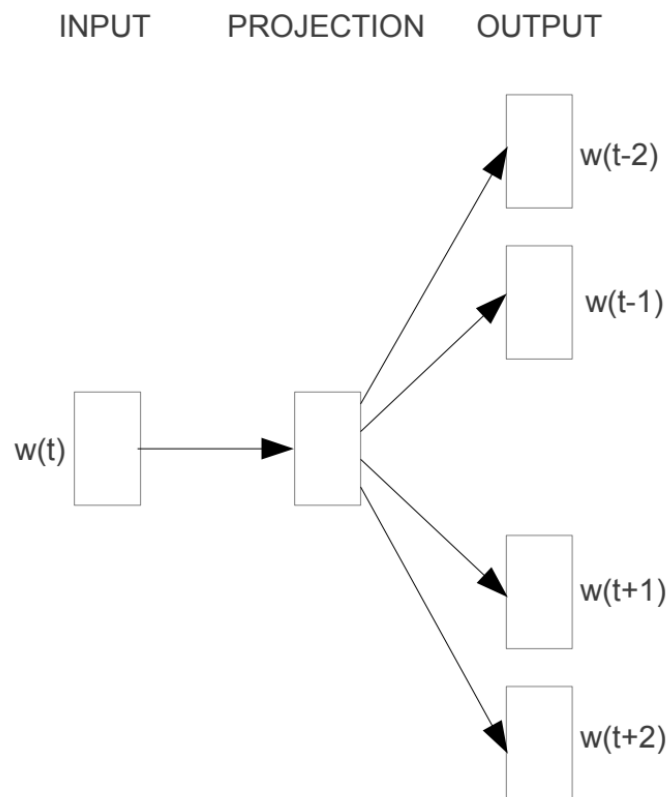
词表示 (Word Representation)

- **分布式表示** (distributed representation)
 - ✓ 基于矩阵的分布表示
 - ✓ 基于神经网络的分布表示
- **核心思想:**
 - 选择一种方式描述上下文
 - 选择一种模型刻画目标词与其上下文之间的关系

word2vec



CBOW (介绍)

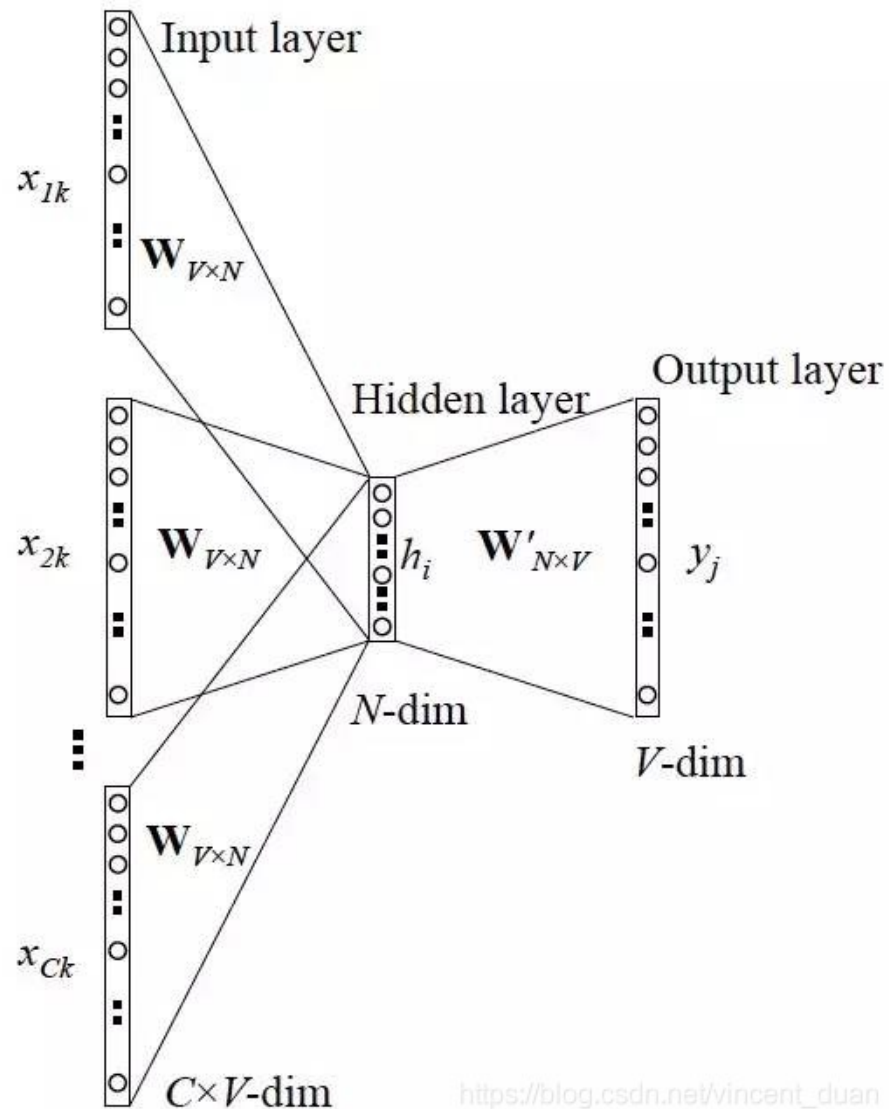


Skip-gram (略)

• CBOW模型

- ① 当前词的上下文词语的one-hot编码输入到输入层
- ② 分别乘以同一个矩阵 W 后分别得到各自的 $1 \times N$ 向量
- ③ 将这些 $1 \times N$ 向量取平均为一个 $1 \times N$ 向量
- ④ 将这个 $1 \times N$ 向量乘矩阵 W' ，变成一个 $1 \times V$ 向量
- ⑤ 做Softmax分类，与真实标签 $1 \times V$ 向量计算交叉熵损失
- ⑥ 每次前向传播之后反向传播误差，调整矩阵 W 和 W' 的值

$$\mathcal{L}(\theta) = \prod_{t=1}^T P(x_t | \{x_{t+j}, j \in [-m, m]\})$$
$$\bar{v}_t = \frac{1}{2m} \sum_{j \in [-m, m]} v_{t+j}, \quad \bar{v}'_t = W'^T \bar{v}_t$$
$$P(x_t | \{x_{t+j}\}) = \frac{\exp(u_{x_t}^T \bar{v}'_t)}{\sum_{k \in V} \exp(u_k^T \bar{v}'_t)}$$



https://blog.csdn.net/vincent_duan

word2vec



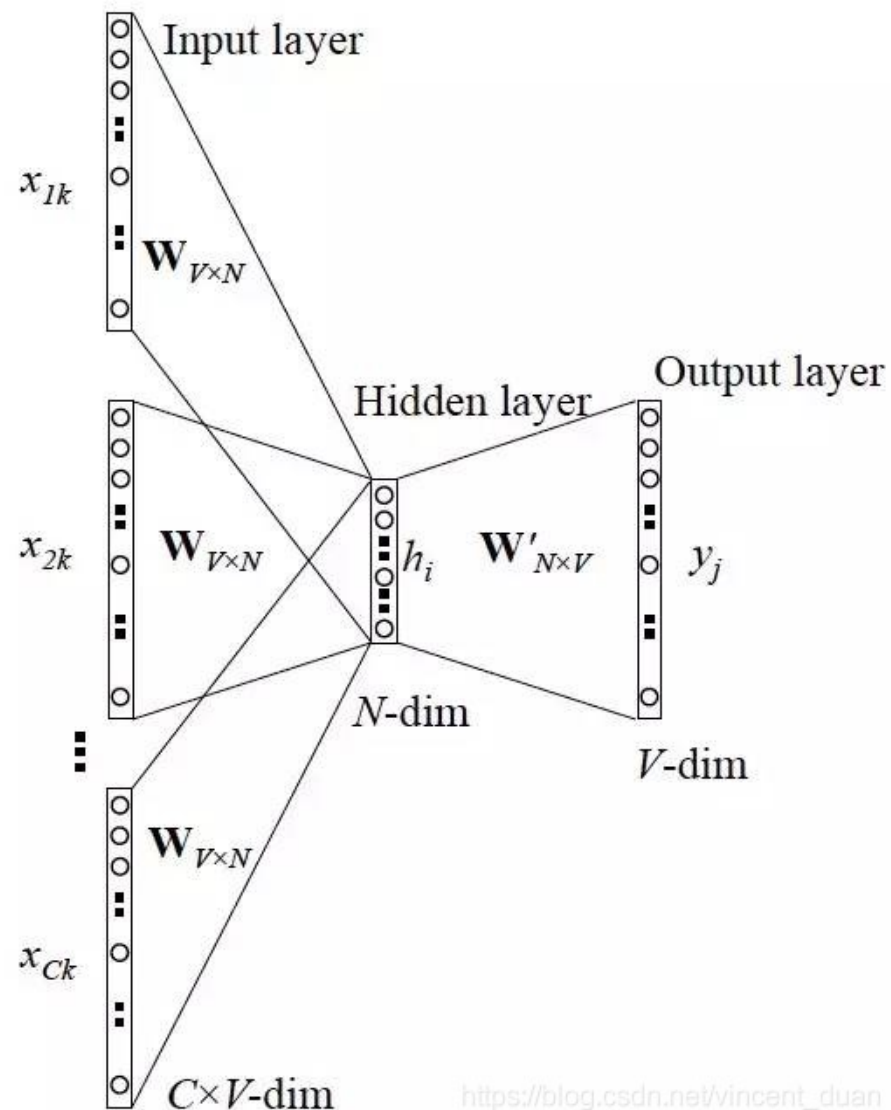
• CBOW模型

给定词的
 $1 \times V$ 独热向量

周围词向量矩阵
 $W_{V \times N}$

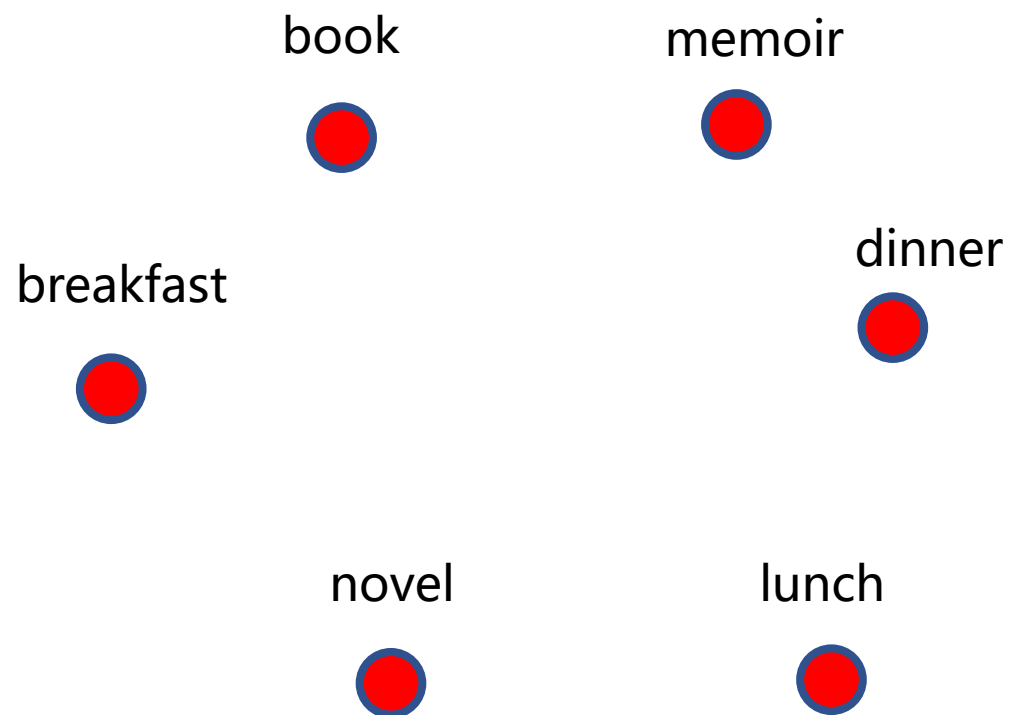
学到的 $1 \times N$
词向量表示

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 17 & 24 & 1 \\ 23 & 5 & 7 \\ 4 & 6 & 13 \\ 10 & 12 & 19 \\ 11 & 18 & 25 \end{bmatrix} = \begin{bmatrix} 10 & 12 & 19 \end{bmatrix}$$



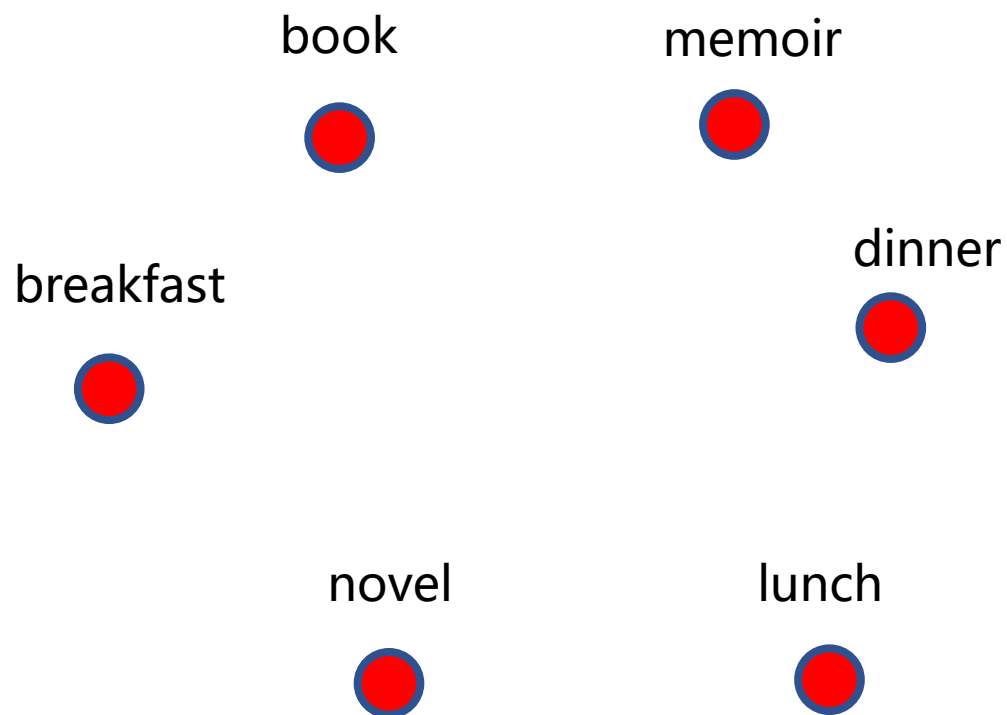
https://blog.csdn.net/vincent_duan

word2vec



独特表示的词特征空间示例

word2vec



自监督学习后的word2vec词特征空间示例

word2vec



自监督学习后的word2vec词特征空间示例

word2vec——词向量示例



```
>>> words["city"]
array([[ 0.231087, -0.238098,  0.584713, -0.524351,  0.40278 ,  0.148448,
        0.386096, -0.493994, -0.198922, -0.411161,  0.556962,  0.220978,
        -0.304637, -0.499713, -0.092555,  0.262613,  0.752704,  0.463667,
        0.054477,  0.155809, -0.195134, -0.009269,  0.378139, -0.651306,
        -0.029372, -0.563472,  0.024709,  0.366842, -0.476904, -0.42565 ,
        -0.094642, -0.052822,  0.124612,  0.296046, -0.244881,  0.195957,
        0.223666,  0.064116,  0.577874,  0.083096, -0.378262,  0.196044,
        -0.220993, -0.630213, -0.311214,  0.435611,  0.351486,  0.342794,
        -0.229961, -0.157521,  0.204315,  0.253944, -0.562277, -0.534482,
        -0.4158 , -0.120161,  0.649395, -0.227012, -0.130488, -0.332326,
        -0.691952, -0.400436,  0.410125,  0.026237, -0.408483,  0.188236,
        0.130957, -0.320686,  0.225932, -0.171665, -0.335107, -0.009982,
        -0.680831, -0.023788, -0.165798,  0.345986, -0.232295,  0.021137,
        0.08515 , -0.24387 , -0.142469, -0.058325,  0.086046, -0.173068,
        0.198108,  0.009103,  0.381725,  0.095911,  0.317972, -0.10012 ,
        0.143178,  0.106724, -0.419844, -0.175785, -0.251805,  0.211927,
        0.411175,  0.317378,  0.450316, -0.252661])
```

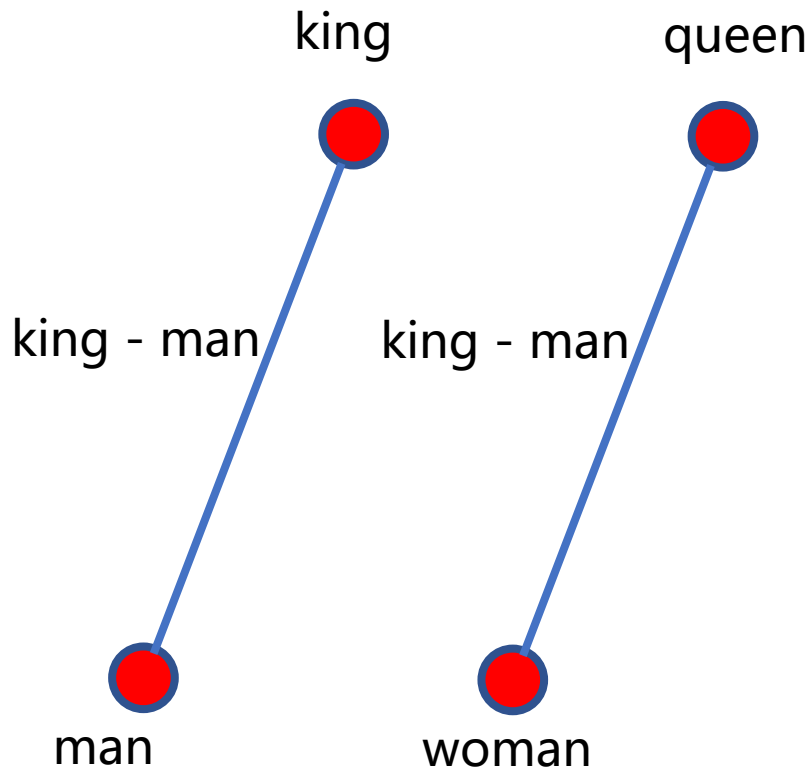
```
>>> distance(words["book"], words["book"])
0
>>> distance(words["book"], words["breakfast"])
0.6351827719357863
>>> distance(words["book"], words["novel"])
0.3436623421719047
>>> distance(words["lunch"], words["breakfast"])
0.2006302059301045
```

```
>>> closest_words(words["book"])[ :6]
['book', 'books', 'essay', 'memoir', 'essays', 'novella']
>>> closest_words(words["lunch"])[ :6]
['lunch', 'dinner', 'lunches', 'snack', 'meal', 'brunch']
```

```
>>> closest_word(words["woman"] + words["king"] - words["man"])
'queen'
>>> closest_word(words["england"] + words["paris"] - words["france"])
'london'
```

```
>>> closest_word(words["japan"] + words["beijing"] - words["china"])
'tokyo'
>>> closest_word(words["hospital"] + words["teacher"] - words["school"])
'nurse'
>>> closest_word(words["hospital"] + words["student"] - words["school"])
'hospital'
```

```
>>> closest_words(words["hospital"] + words["student"] - words["school"])
['hospital', 'patient', 'icu', 'nurse', 'nurses', 'transplant', 'gyn', 'neurosurgery', 'inpatient', 'medical']
```



- 词袋模型 (Bag-of-Words Model)
- 朴素贝叶斯 (Naive Bayes) 模型
- 信息检索, tf-idf
- 词表示 (one-hot, word2vec)

预告：基于神经网络的自然语言处理、Transformer

谢谢



北京大学
PEKING UNIVERSITY

